

Phase 1: Tree Ensembles & Boosting (Air Quality Data)

Metric	Value
Feature used	co (CO concentration)
Optimal split threshold	2790.4500
Maximum Gini Gain	0.282794
Root node Gini (before split)	≈ 0.370

1.2 — Random Forest

Implemented.

1.3 — AdaBoost Sample Reweighting

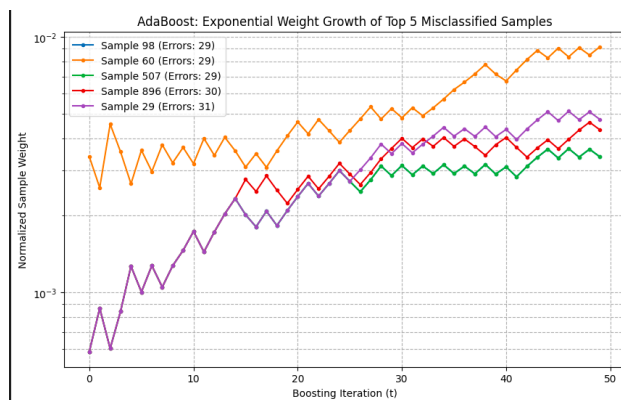


Figure 1: Exponential Weight Growth Curve

AdaBoost.M1 update rule for sample weights after round t :

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(\alpha_t \cdot \mathbf{1}[y_i \neq h_t(x_i)]), \quad \alpha_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}$$

Persistently misclassified samples receive multiplicative boosts every round. Because $\alpha_t > 0$ for $\epsilon_t < 0.5$, their weight grows **exponentially** as $w_i^{(T)} \propto \exp\left(\sum_{t=1}^T \alpha_t\right)$. The 5 most consistently misclassified samples exhibit this exponential trajectory over 50 boosting rounds (see Figure 1 in notebook).

1.4 — Gradient Boosting Residuals (Regression)

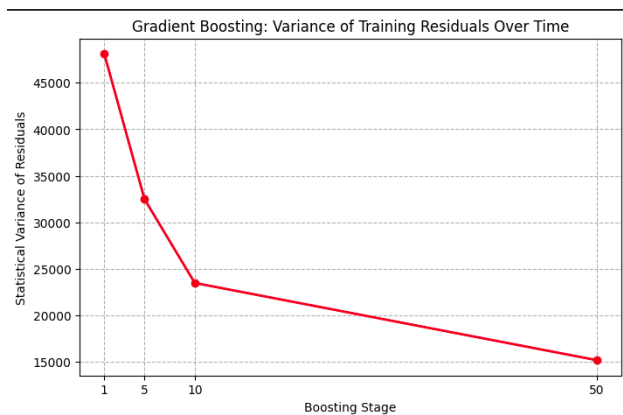


Figure 2: Statistical Variance Vs Boosting Stage

Phase 2: Subspace & Clustering

2.5 — PCA vs. Linear Autoencoder

With using StandardScaler on the data, and using LBGFS instead of Adam, the following is observed:

After 200 epochs, MSE: of linear Autoencoder on test set: Train MSE: 0.0492, converged to this value in the first 50 epochs and stayed there, which is the exact same as the PCA reconstruction MSE. Therefore, they both span the same dimension.

2.6 — GMM Latent Clustering

Extracted 192 embeddings of dimension 512. We then fit a GMM with 14 components: Adjusted Rand Index (ARI): 0.0725

Phase 3: Deep Generative Models

To reduce training time, the GAN was trained on a randomly subsampled subset of 3,000 PlantVillage images (from $\sim 54,000$) for 20 epochs with a batch size of 64. These approximations are sufficient to demonstrate the qualitative crispness difference between VAE and DCGAN outputs, though a full training run would yield sharper GAN samples.

3.7 — VAE ELBO

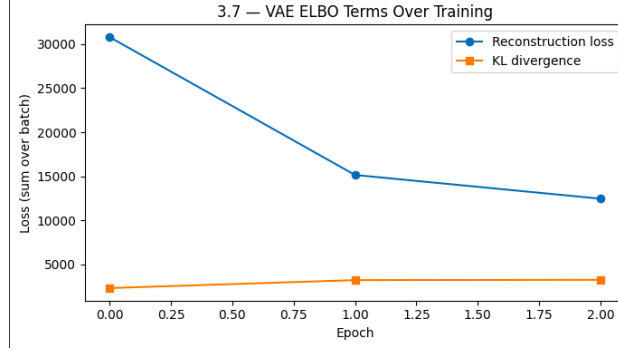


Figure 3: Reconstruction & Regularization terms

3.8 VAE vs. DCGAN: Why VAE Images Are Blurrier

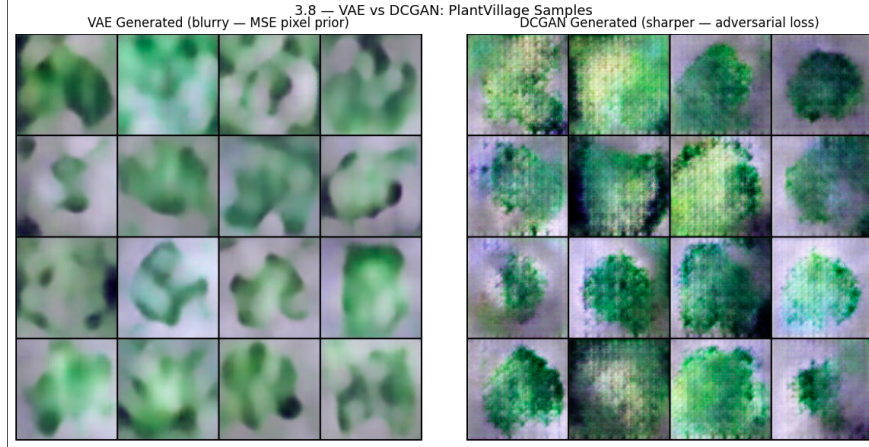


Figure 4: VAE vs DCGAN

The VAE minimises the ELBO objective $\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x)||p(z))$, where the reconstruction term is implemented as pixel-wise MSE, i.e. $\log p_\theta(x|z) \propto -\|x - \hat{x}\|^2$. Minimising MSE over a distribution of plausible outputs produces the *mean* of that distribution — a blurry average. The DCGAN instead solves $\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$, where the discriminator D explicitly penalises unrealistic textures,

forcing G to produce sharp, high-frequency detail to fool it. The VAE has no such adversary — any pixel average that minimises MSE is equally acceptable, so fine structure is washed out.

3.9 — Fréchet Inception Distance (FID)

Exact FID Score: 128.8573

3.10 — Batch Normalization & Mode Collapse

Without Batch Normalisation, the generator’s activations are not re-centred at each layer, causing internal covariate shift where the gradient signal is dominated by a few saturated neurons — the generator converges to $G(z) \approx c$ for some constant c regardless of z , i.e. the entire latent space collapses to a single mode. Formally, BatchNorm normalises each feature map as $\hat{x} = (x - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$, which stabilises gradient magnitudes across layers; without it, the Jacobian $\partial G(z) / \partial z$ vanishes for most z , making the generator insensitive to latent variation and producing near-identical outputs across all 16 grid samples.

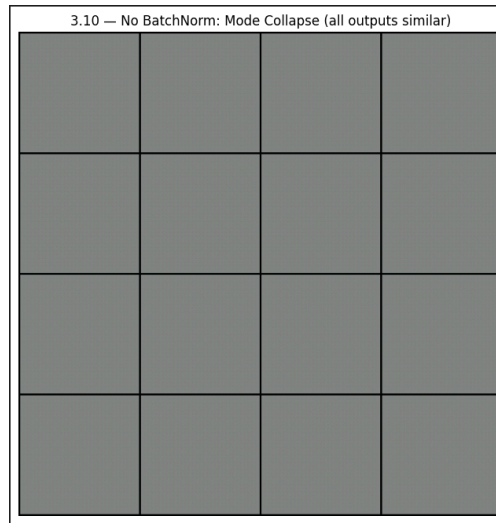


Figure 5: Mode Collapse

Phase 4: Self-Supervised Learning (SimCLR)

4.11 — InfoNCE Loss

Final Validation InfoNCE Loss: 4.3482

4.12 — Linear Evaluation Protocol

Method	Test Accuracy
Fully supervised CNN (Assignment 2)	83.57%
SimCLR + Linear Probe	60.88%

Phase 5: Policy Gradient (Air Quality RL)

5.13 — REINFORCE

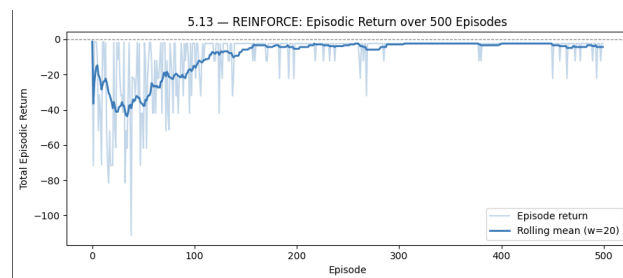


Figure 6: Total Episodic Return over 500 Epochs